Experiment Title: Solving Two water jug Problem.

Aim / Objective: Write a program to solve the 2-water jug problem by manually providing moves from the user.

Theory: The Two Water Jug Problem is a state-space problem. It has two Jugs with maximum capacities and unlimited water supply. Initial state (capacity of jug 1, capacity of jug 2) will be (0, 0) i.e. both jugs will be empty in the beginning. The aim is to reach the goal state with the target volume of water, obtaining the optimized solution for it.

Constraints of the problem:

- Only two jugs x and y can be used to solve the problem.
- We assume that the jugs have integer capacities.
- Both the jugs have no markings, so only the maximum capacity of both the jugs is known.
- No water can be spilled out of the jug.
- The target volume has to be less than or equal to the sum of maximum capacities of the jugs.
- Pour water from one jug into another until the receiving jug is full, or the transferring jug is empty.
- The user has to enter the action (Fill, Empty, Pour) till the target is achieved accurately.

To Implement the problem we need following series of operations:

1. Fill x: Fill x to maximum capacity
2. Fill y: Fill y to maximum capacity
3. Empty x: Empty jug x
4. Empty y: Empty jug y
5. Pour x →y: Pour the water from jug x to jug y
6. Pour y →x: Pour the water from jug y to jug x

Algorithm/ Methodology: ***Two_Water_Jug (x, y, target):***

1. *Read the capacity of jug1, jug2 (capx, capy) and target (x', y')*
2. *while (x,y) != target do :*

    Read the action from user

a. *Fill x : x = capx*
b. *Fill y : y = capy*
c. *Empty x : x = 0*
d. *Empty y : y = 0*
e. *Pour x →y : pour = min(x, capy-y), x = x-pour, y = y+pour*
f. *Pour y →x : pour = min(y, capx-x), y = y-pour, x = x+pour*

3. 3. *Print (x,y)*

Dataset Description: Capacity of jug 1 : capx

Capacity of jug 2 : capy

Current capacity of jug 1 : x

Current capacity of jug 2 : y

Target need to be achieved : (x′, y′)

All the parameters are **integer input from user** with constraints **0 ≤ x ≤ capx** and **0 ≤ y ≤ capy**

Implementation:

```
# user inputs
capx=int(input("Enter the capacity of X: "))

capy=int(input("Enter the capacity of Y: "))

target=[int(input("Enter the target value of X: ")),int(input("Enter the target value of Y: "))]

# Two water jug problem function

def twowaterjug(x,y,target):

  while [x,y] != target :

     ip=input("Enter the action number \n 1. Fill X \n 2. Fill Y \n 3. Empty X \n 4. Empty Y \n 5. Pour X to Y \n 6. Pour Y to X \n")

    match ip:

      case '1':

        x=capx

      case '2':

        y=capy
```

```
        case '3':
          x=0
        case '4':
          y=0
        case '5':
          pour=min(x,capy-y)
          x=x-pour
          y=y+pour
        case '6':
          pour=min(y,capx-x)
          y=y-pour
          x=x+pour
  print(x,y)
twowaterjug(0,0,target)
```

Output / Result:

```
... Enter the target value of Y: 4
    Enter the action number
     1. Fill X
     2. Fill Y
     3. Empty X
     4. Empty Y
     5. Pour X to Y
     6. Pour Y to X
    2
    Enter the action number
     1. Fill X
     2. Fill Y
     3. Empty X
     4. Empty Y
     5. Pour X to Y
     6. Pour Y to X
    6
    Enter the action number
     1. Fill X
     2. Fill Y
     3. Empty X
     4. Empty Y
     5. Pour X to Y
     6. Pour Y to X
    3
    Enter the action number
     1. Fill X
     2. Fill Y
     3. Empty X
     4. Empty Y
     5. Pour X to Y
     6. Pour Y to X
    6
```

```
Enter the action number
 1. Fill X
 2. Fill Y
 3. Empty X
 4. Empty Y
 5. Pour X to Y
 6. Pour Y to X
2
Enter the action number
 1. Fill X
 2. Fill Y
 3. Empty X
 4. Empty Y
 5. Pour X to Y
 6. Pour Y to X
6
Enter the action number
 1. Fill X
 2. Fill Y
 3. Empty X
 4. Empty Y
 5. Pour X to Y
 6. Pour Y to X
3
0 4
```

Result Analysis: Let us assume capacity_x = 3 and capacity_y = 5

Goal State to achieve is (x,y) = (0,4)

1. Fill y { (x,y) = (0,5) }
2. Pour y →x { (x,y) = (3,2) }
3. Empty x { (x,y) = (0,2) }
4. Pour y →x { (x,y) = (2,0) }
5. Fill y { (x,y) = (2,5) }
6. Pour y →x { (x,y) = (3,4) }
7. Empty x { (x,y) = (0,4) }

The result is obtained in 7 steps from python code from the same series of operations. The code is tested and verified for the manual action selected by the user to solve the two jugs problem.

Conclusion:

- The Two Jugs Problem is successfully implemented in python.
- All the constraints are satisfied in the implementation.
- (x ,y) has discrete states in the state space.
- The starting point is always (0, 0).
- Manual moves provided by the user shows the decision making.

Experiment Title: The Missionaries and Cannibals problem.

Aim / Objective: Write a program to implement the Missionaries and Cannibals problem.

Theory: The Missionaries and Cannibals problem is a state-space problem. Three missionaries and cannibals are supposed to cross the river to another bank. We will have states in the form of (number of missionaries, number of cannibals, boat position). Initial state is (3, 3, 0) and goal state is (0, 0, 1) where boat position 0 means the left bank and 1 means the right bank. The aim is to achieve a goal state with optimized steps.

Constraints of the problem:

- All the three missionaries and cannibals have to cross the river.
- Boat maximum capacity is two.
- At least one missionary or cannibal has to return with a boat for the next transfer.
- The number of missionaries must be greater than or equal to the number of cannibals on both the banks, else missionaries are consumed by cannibals (assuming missionaries are present).
- The number of missionaries or cannibals can be zero on either side of the river.
- No user input, solution obtained by machine.

Possible combinations to move (m, c) from one bank to another:

(0, 1), (1, 0), (1, 1), (0, 2), (2, 0). Each node in the tree will differ by these combinations.

BFS (Breadth first search) is used to solve the problem, BFS uses queue data structure for traversal. BFS algorithm used to get optimal number of crossings.

Algorithm / Methodology: ***BFS_Missionaries_Cannibals:***

Input: Initial State (S)=(3,3,0), Target State (T)=(0,0,1)

Output: Optimal Path from Initial State to Target State

1. *Queue Q ← { S, [S] }, Visites set V ← {S}*
2. *While Q ≠ ∅ do:*

   (u, path) = Dequeue(Q)

   If u = T return path

   For each valid move $\Delta = (\Delta_m, \Delta_c) \in \{(0, 1), (1, 0), (1, 1), (0, 2), (2, 0)\}$ do:

   Let u = (m,c,b)

If b=0 then $v \leftarrow (m-\Delta_m, c-\Delta_c, 1)$

Else $v \leftarrow (m+\Delta_m, c+\Delta_c, 0)$

If $v \notin V$ *isValid(v)*

$V \leftarrow V \cup \{v\}$

Enqueue(Q, path $\cup \{v\}$*)*

3. *Return* $\emptyset$

Dataset Description: start = (3,3,0)

target = (0,0,1)

valid moves = { (0, 1), (1, 0), (1, 1), (0, 2), (2, 0) }

No user input required

All the values are of Integer type

Implementation:

```
totalM = 3
totalC = 3
valid_moves = [(0,1),(1,0),(1,1),(0,2),(2,0)] # As max boat capacity is
2 these are possible combinations

def valid_check(m, c): # check constraints
    if not (0 <= m <= totalM and 0 <= c <= totalC):
        return False
    if 0 < m < c:
        return False
    otherm, otherc = totalM - m, totalC - c
    if 0 < otherm < otherc:
        return False
    return True

def get_nodes(state): #Get all valid next nodes
    m, c, b = state
    next_nodes = []
    for move in valid_moves:
        if b == 0:
            new_m, new_c, new_b = m - move[0], c - move[1], 1
        else:
            new_m, new_c, new_b = m + move[0], c + move[1], 0
        if valid_check(new_m, new_c):
            next_nodes.append((new_m, new_c, new_b))
    return next_nodes
```

```python
def solve_m_c(): # find best possible path using BFS
    start = (3, 3, 0)
    target = (0, 0, 1)
    queue = [(start, [start])]
    visited = {start}
    while queue:
        current, path = queue.pop(0)
        if current == target:
            return path
        for neighbor in get_nodes(current):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
    return None

print("Path: ",solve_m_c()) #print path
```

Output / Result:

```
Path:  [(3, 3, 0), (2, 2, 1), (3, 2, 0), (3, 0, 1), (3, 1, 0), (1, 1, 1), (2, 2, 0), (0, 2, 1), (0, 3, 0), (0, 1, 1), (0, 2, 0), (0, 0, 1)]
```

Result Analysis: The result we get is:

Path: [(3, 3, 0), (2, 2, 1), (3, 2, 0), (3, 0, 1), (3, 1, 0), (1, 1, 1), (2, 2, 0), (0, 2, 1), (0, 3, 0), (0, 1, 1), (0, 2, 0), (0, 0, 1)]

It has 12 states and 11 moves, this is the optimal solution as BFS searches the target level by level, it makes the path efficient.

Visited nodes skipping helps to reduce searching time.

Time Complexity is O(V+E)

Viva Questions:

1. **Why is BFS preferred over DFS?**

BFS explores neighbour nodes whereas DFS explores a branch of tree first. DFS hence doesn't guarantee optimal solutions (in such cases). DFS will cost more crossings than BFS as it prefers depth over breadth, and here we wish to optimize the number of crossings. Hence BFS is preferred here.

2. **What happens if boat capacity is increased to 3?**

The whole solution changes as moves that are possible changes therefore the tree will change. As capacity increases the number of crossings will decrease here the boast can move like: (3,3,0), (3,0,1), (3,1,0), (0,1,1), (1,1,0), (0,0,1) [This might not be optimal solution but one of the solutions).

3. **Can the water jug problem be solved using BFS instead of manual input? How?**
   Yes, The two water jug problem can be solved using BFS allowing no user input. Foe each state we will find the next possible state with the operations fill, empty, pour and try to match with the target state, this will ensure the minimum number of operations required to reach the target state. Here we will also use the visited set to reduce searching time.
4. **How do the constraints prune the invalid states?**

Constraint	Pruning Action	Resulting Benefit
Safety	Removes states where M < C (if M>0)	Ensures only allowed paths are followed.
Boundary	Removes states where counts < 0 or > 3	Keeps the state space finite.
Visited Set	Removes states already explored	Prevents infinite loops and cycles.
Capacity	Limits moves to 1 or 2 people	Reduces the branching factor per node.

Conclusion:

- The BFS choice is beneficial over DFS, Minimum number of crossings are required using BFS.
- Visited Set reduced the computational overhead significantly.

- No user input required.
- Constraints are implemented correctly.